Memory

Arrays, argmax and state machines without indexing, comparing or branching

---

Memory is a one-hot vector

A Python program reads and writes memory with an *index*: `xs[i]`. A network has no indexing instruction. It has vectors, matrices and ReLU, and nothing else. So how can a network "look something up"?

The trick is to turn the address into a vector with exactly one `1` in it, a **one-hot** vector. For example, with four cells the address $i = 2$ becomes

$$\text{onehot}(2) = (0, 0, 1, 0).$$

The one-hot vector is the address. Reading and writing then become arithmetic with it:

- **read:** multiply each cell by its one-hot bit and add everything up. Cells whose bit is 0 vanish.
- **write:** use the bit to decide, cell by cell, "take the new value" or "keep the old one".

There are two cases, and they cost very differently:

memory is…	**example**	**how the lookup works**
a **constant** table, known when you design the network	a lookup table of costs, a graph, a DFA transition table	`onehot @ table`: an ordinary matrix product. **The weights are the memory.** No big-M needed.
runtime data, an input or a computed value	the array in `xs[i]`, the open list in A*	`sum_j select(onehot_j, xs[j])`. The bit and the cell are both runtime values, so each product needs a big-M select.

Chapters 4 and 5 use both kinds: the graph is constant memory (weights), while "how far have we got" is runtime memory (activations).

This chapter builds the machinery: reading (example 10), writing (11), finding the best cell (12, `argmax`) and a machine that carries a one-hot *state* through a string (13).

Key idea

Address = one-hot vector. Everything in this chapter is "make a one-hot, then use it". A constant memory is read with a matrix product; a runtime memory needs a big-M select per cell, so reading N cells costs N selects.

10 Array read

basic_python/03_memory/10_array_read.py

Problem. Given a list of four integers `xs` (each in $[0, 7]$) and an index `i` in $[0, 3]$, compute `xs[i]` with no indexing and no branching. Both `xs` and `i` are only known at runtime.

A · control flow

```
def control_flow(xs, i):
    return xs[i]
```

The machine jumps straight to cell `i`.

B · branchless arithmetic

```
def branchless(xs, i):
    oh = onehot(i)
    total = 0
    for j in range(N):
        total += select(oh[j], xs[j])
    return total
```

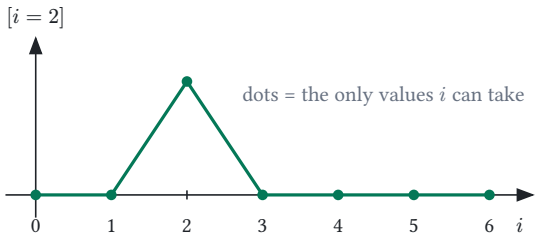
Touch **every** cell; all but one contribute 0.

Step 1: the address as a one-hot

For one cell j we need $[i = j]$. From the toolbox, $a = b \Leftrightarrow 1 - \text{step}(a - b) - \text{step}(b - a)$, and $\text{step}(x) = \text{relu}(x) - \text{relu}(x - 1)$. Written out,

$$[i = j] = 1 - \underbrace{\text{relu}(i - j)}_{r_0} + \underbrace{\text{relu}(i - j - 1)}_{r_1} - \underbrace{\text{relu}(j - i)}_{r_2} + \underbrace{\text{relu}(j - i - 1)}_{r_3}.$$

So each cell needs four ReLU neurons. As a function of a real i the result is a "tent" with its peak at $i = j$. Because i is an integer, we only ever sample the tent at whole numbers, where it is exactly 0 or 1:



Worked example for $i = 2$ (the four neuron outputs per cell, then the result):

cell j	$\text{relu}(i - j)$	$\text{relu}(i - j - 1)$	$\text{relu}(j - i)$	$\text{relu}(j - i - 1)$	$1 - r_0 + r_1 - r_2 + r_3$	bit
0	2	1	0	0	$1 - 2 + 1 - 0 + 0$	0
1	1	0	0	0	$1 - 1 + 0 - 0 + 0$	0
2	0	0	0	0	$1 - 0 + 0 - 0 + 0$	1
3	0	0	1	0	$1 - 0 + 0 - 1 + 0$	0

Step 2: keep one cell, kill the rest

Now `xs[j]` times its bit. Both are runtime values, so we cannot multiply them. Use the big-M select from the toolbox:

$$\text{select}(m, x) = \text{relu}(x - M(1 - m)) - \text{relu}(-x - M(1 - m)).$$

If $m = 1$ the shift $M(1 - m)$ is 0 and we get $\text{relu}(x) - \text{relu}(-x) = x$. If $m = 0$ both ReLUs see $\pm x - M \le 0$ (this needs $|x| \le M$) and the result is 0. Here $M = 7$ because cells are in $[0, 7]$.

Trace, `xs = (5, 0, 7, 2)`, $i = 2$, bits $(0, 0, 1, 0)$:

j	bit m	$x =$ `xs[j]`	$\text{relu}(x - 7(1 - m))$	$\text{relu}(-x - 7(1 - m))$	select
0	0	5	$\text{relu}(-2) = 0$	$\text{relu}(-12) = 0$	0
1	0	0	$\text{relu}(-7) = 0$	$\text{relu}(-7) = 0$	0

j	bit m	$x =$ xs[j]	relu$(x - 7(1-m))$	relu$(-x - 7(1-m))$	select
2	1	7	relu$(7) = 7$	relu$(-7) = 0$	7
3	0	2	relu$(-5) = 0$	relu$(-9) = 0$	0

The sum is $0 + 0 + 7 + 0 = 7 =$ xs[2]. Correct.

> **Why M must be big enough**
>
> The select only kills a masked-off cell if $x - M \le 0$. With $M = 4$ and a cell holding 7, a "masked off" cell would leak relu$(7 - 4) = 3$ into the sum. **Rule:** M is any upper bound on $|x|$. Bigger is safe but not free in a real network (it makes weights large). So always state the bound in a comment and derive it from the input range.

Step 3: the network

Because xs ≥ 0, the second half relu$(-x - M(1-m))$ is always 0 and we can drop it. That leaves **three layers**. The input is $x = (i, \text{xs}[0], \ldots, \text{xs}[3])$.

- **Layer 1** (21 neurons): per cell, the four tent ReLUs, plus relu(xs$[j]$) to carry the data forward (exact, since xs$[j] \ge 0$).
- **Layer 2** (4 neurons): $a_j = \text{relu}\big(\text{xs}[j] + M \cdot \text{bit}_j - M\big)$. Substituting $\text{bit}_j = 1 - r_0 + r_1 - r_2 + r_3$, the constants cancel and what is left is a plain weighted sum:

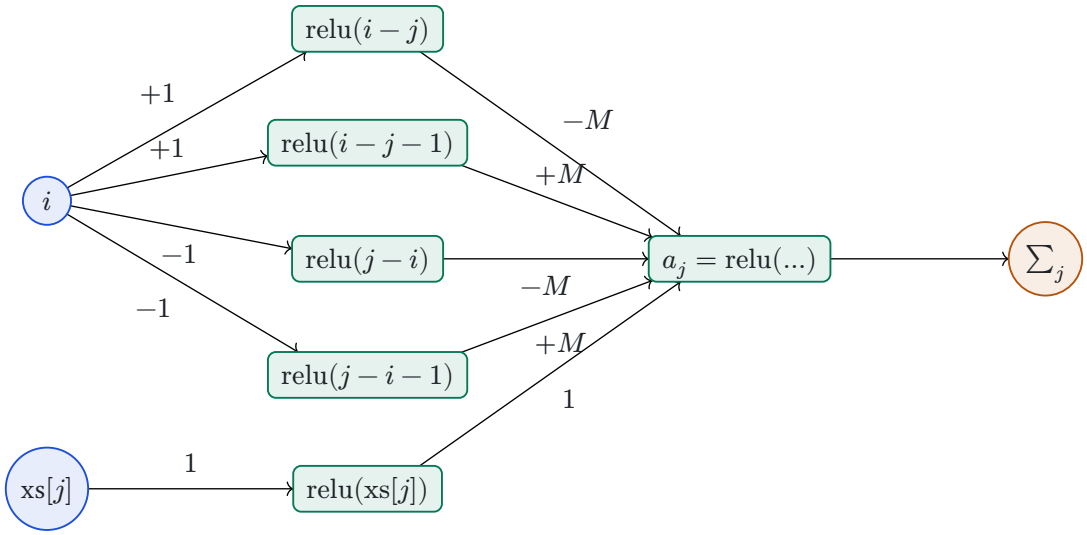
$$a_j = \text{relu}(-Mr_0 + Mr_1 - Mr_2 + Mr_3 + \text{xs}[j]).$$

- **Layer 3:** $y = \sum_j a_j$ (a row of ones).

For cell $j = 2$ the weights of layer 1 (columns are $i, \text{xs}_0, \ldots, \text{xs}_3$) and of layer 2 are:

$$W_1^{(2)} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \end{pmatrix}, \quad b_1^{(2)} = \begin{pmatrix} -2 \\ -3 \\ 2 \\ 1 \end{pmatrix}, \quad W_2^{(2)} = (-7 \;\; 7 \;\; -7 \;\; 7 \;\; 1) \text{ (over } r_0..r_3, \text{relu}(\text{xs}_2)\text{)}$$

The full W_1 is five such blocks stacked (cells 0..3 and the relu(xs) carries); W_2 has one such row per cell; W_3 is $(1, 1, 1, 1)$. In the code they are filled in by a loop at import time.



The wiring for **one** cell j. The real network has four of these side by side, all fed by the same i, all feeding the same final sum.

Contrast: a constant table is just a matrix

Suppose the array is not an input but fixed forever, say `TABLE = [3, 1, 4, 7]`. Then $\text{table}[i] = \text{onehot}(i) \cdot \text{TABLE}$, and since the one-hot is affine in layer 1 and TABLE is a constant row, the whole read is **one layer of ReLUs followed by affine maps**. There is no second layer, no M, no select:

$$y = \text{TABLE} \cdot (\mathbf{1} + Oh_1),$$

where h_1 is the layer-1 output and O is the constant $N \times 5N$ matrix that has the row $(-1, 1, -1, 1)$ in each cell's four columns.

Compare the costs for N cells:

	neurons	layers of ReLU	needs big-M
constant table	$4N$	1	no
runtime array	$5N + N$	2	yes, $M \geq$ bound of the values

The code checks both. That's why graphs and cost tables in the next chapters become **weights**, while only things that change during the computation need selects.

> **Pitfall: out of range**
>
> For $i = 7$ every tent $[i = j]$ is 0, so the result is silently 0. A network can't raise an `IndexError`. If an address might be invalid, decide what `0` should mean and make sure that is acceptable.

11 Array write

basic_python/03_memory/11_array_write.py

Problem. `xs[i] = v`, then use the list. Data flow has no "modify in place"; it returns a **new** list. Each cell j becomes `v` if j is the address, else keeps its old value.

A · control flow

```
def control_flow(xs, v, i):
    xs = list(xs)
    xs[i] = v
    return xs
```

B · branchless arithmetic

```
def branchless(xs, v, i):
    oh = onehot(i)
    new = []
    for j in range(N):
        new.append(
            xs[j] + select(oh[j], v -
xs[j]))
    return new
```

Derivation

For one cell, "v if m else x" is the toolbox's `x if m else y` row:

$$\text{new}_j = x_j + \text{select}\big(m_j, v - x_j\big), \quad m_j = [i = j].$$

Check both cases: $m_j = 1$: $x_j + (v - x_j) = v$. $m_j = 0$: $x_j + 0 = x_j$.

The thing being selected is the **difference** $d_j = v - x_j$. It is a runtime value that can be negative, so here we need **both** ReLUs of the select, and M must bound $|v - x_j|$. With values in $[0, 5]$ the difference is in $[-5, 5]$, so $M = 5$.

Trace, `xs = (5, 0, 3, 2)`, $v = 4$, $i = 2$, bits $(0, 0, 1, 0)$:

j	m_j	$d_j = v - x_j$	$\text{select}(m_j, d_j)$	$x_j + \text{select}$	new
0	0	$4 - 5 = -1$	$\text{relu}(-1 - 5) - \text{relu}(1 - 5) = 0$	$5 + 0$	5
1	0	$4 - 0 = 4$	$\text{relu}(4 - 5) - \text{relu}(-4 - 5) = 0$	$0 + 0$	0
2	1	$4 - 3 = 1$	$\text{relu}(1) - \text{relu}(-1) = 1$	$3 + 1$	4
3	0	$4 - 2 = 2$	$\text{relu}(2 - 5) - \text{relu}(-2 - 5) = 0$	$2 + 0$	2

Result $(5, 0, 4, 2)$. Only cell 2 changed.

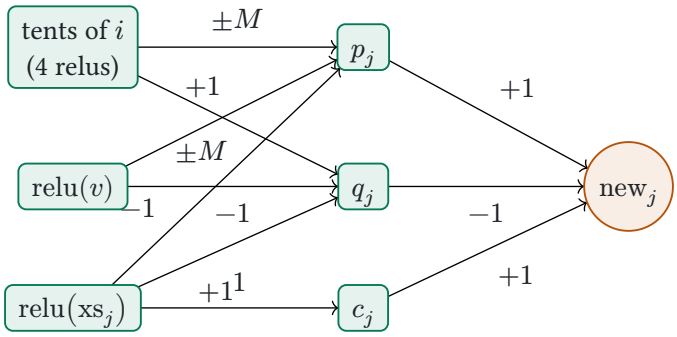
The network

Input $x = (i, v, \text{xs}_0, \ldots, \text{xs}_3)$, three layers:

- **Layer 1** ($5N + 1 = 21$ neurons): the four tent ReLUs per cell, plus $\text{relu}(v)$ and $\text{relu}(\text{xs}_j)$ as carried copies (everything is ≥ 0 so the copy is exact).
- **Layer 2** ($3N = 12$ neurons): for each cell,

$$p_j = \text{relu}\big((v - \text{xs}_j) + M\text{bit}_j - M\big), \quad q_j = \text{relu}\big(-(v - \text{xs}_j) + M\text{bit}_j - M\big), \quad c_j = \text{relu}\big(\text{xs}_j\big).$$

  With $\text{bit}_j = 1 - r_0 + r_1 - r_2 + r_3$ the constants cancel, so p_j's weight row is $(-M, M, -M, M)$ on the four tent neurons, $+1$ on $\text{relu}(v)$ and -1 on $\text{relu}(\text{xs}_j)$. The q_j row is the same with the last two signs flipped.
- **Layer 3** (affine only): $\text{new}_j = c_j + p_j - q_j$.



The wiring for **one** cell j; the "tents" box stands for the four neurons of example 10.

Cost: read-modify-write

A write touches **every** cell. For N cells you pay N equality tests and N selects (about $5N$ neurons in layer 1 and $3N$ in layer 2), regardless of how little actually changes. In a normal CPU a write is $O(1)$; in a network "addressing" costs a full pass over memory. That is the price of having no pointers, and why we keep arrays tiny (a frontier of at most a dozen nodes, not a million).

> **A cheaper special case**
>
> If the new value is a **constant** the select disappears. For `xs[i] += 1` you get $\text{new}_j = \text{xs}_j + [i = j]$: the one-hot is added directly (no big-M, one layer fewer). The select is only needed when **both** the mask and the value being written are runtime values you must combine.

12 argmax and argmin

basic_python/03_memory/12_argmax.py

Problem. Which cell is biggest? Return a one-hot vector marking it, plus the maximum value. If several cells tie, the **lowest index wins**, exactly like `xs.index(max(xs))` or `max(range(n), key=lambda k: xs[k])` in Python.

A · control flow

```
def control_flow(xs, sign=+1):
    pick = max if sign > 0 else min
    best = pick(range(len(xs)),
                key=lambda k: xs[k])
    return ([1 if k == best else 0
             for k in range(len(xs))],
            xs[best])
```

B · branchless arithmetic

```
for i in range(N):
    bits = 0
    for j in range(N):
        if j == i: continue   # constants
        d = sign * (xs[i] - xs[j])
        bits += step(d) if j < i \
                else step(d + 1)
    onehot.append(relu(bits - (N-2)))
value = sum_i select(onehot[i], xs[i])
```

The idea: every index runs a tournament

Index i is **the** winner exactly when it beats every other index j. "Beats" is defined so that ties are resolved towards the **smaller** index:

opponent	i beats j when…	as a bit
$j < i$ (earlier)	$x_i > x_j$ *strictly*: on a tie the earlier one wins	$\text{step}(x_i - x_j)$
$j > i$ (later)	$x_i \geq x_j$ on a tie i wins, because it is earlier	$\text{step}(x_i - x_j + 1)$

Key idea

Tie-break rule: strict `>` against earlier indices, `>=` against later ones. Then exactly one index wins every contest, and it is the **lowest index among the maxima**. Remember this rule: A* (chapter 4) uses argmin as "pick the best node", and two nodes with equal cost must be resolved the same way Python resolves them, otherwise the network and the algorithm silently disagree.

Why exactly one winner? Let m be the maximum value and k the smallest index holding it. Index k is $\geq$ everything after it and strictly $>$ everything before it (nothing earlier equals m), so it wins. Any other i loses to someone: if $x_i < m$ it loses to k; if $x_i = m$ then $i > k$, and the contest against the earlier k is strict, so i loses.

AND over all opponents

Each index has $n-1$ opponents, so $n-1$ bits. Their AND is 1 only when all are 1. Generalising $\text{AND}(p, q) = \text{relu}(p + q - 1)$:

$$\text{won}_i = \text{relu}\left(\sum_{j \neq i} \text{beats}_{ij} - (n-2)\right).$$

The sum is at most $n-1$; subtracting $n-2$ leaves 1 only if every bit is 1, and ≤ 0 (killed by the ReLU) otherwise.

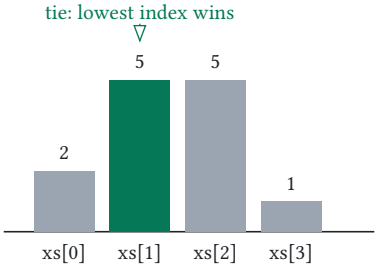
Trace, `xs = (2, 5, 5, 1)`, argmax. Each row is i, each column an opponent j; the entry is the bit "i beats j":

i	x_i	$j=0$	$j=1$	$j=2$	$j=3$	**sum**	**relu(sum − 2)**
0	2	–	$2 \geq 5$: 0	$2 \geq 5$: 0	$2 \geq 1$: 1	1	0
1	5	$5 > 2$: 1	–	$5 \geq 5$: 1	$5 \geq 1$: 1	3	**1**
2	5	$5 > 2$: 1	$5 > 5$: **0**	–	$5 \geq 1$: 1	2	0

i	x_i	$j=0$	$j=1$	$j=2$	$j=3$	**sum**	**relu(sum − 2)**
3	1	$1>2$: 0	$1>5$: 0	$1>5$: 0	–	0	0

The one-hot is $(0,1,0,0)$: index 1 wins the tie against index 2, because index 2's contest against the earlier index 1 is strict ($5>5$ is false). The value is $\sum_i \text{select}(\text{won}_i, x_i) = 0+5+0+0 = 5$ (big-M select with $M=5$).

Argmin is the same machine with the differences negated: $d = -(x_i - x_j)$, with the same strict/non-strict rule. For `xs = (2, 5, 5, 1)` index 3 beats everyone; for an all-equal list `(3, 3, 3, 3)`, index 0 wins in both argmax and argmin.



The network

Three layers, built from loops over the (constant) pairs (i,j):

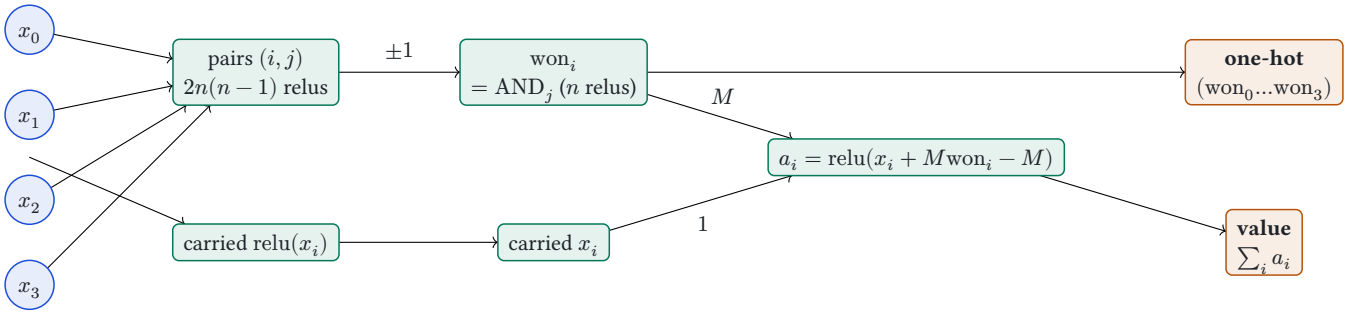
- **Layer 1** ($2n(n-1)+n = 28$ neurons): for each ordered pair, $d = \text{sign}\,(x_i - x_j) + c$ with $c=0$ if $j<i$, $c=1$ if $j>i$; two neurons $\text{relu}(d)$, $\text{relu}(d-1)$ whose difference is $\text{step}(d)$. Plus a carried $\text{relu}(x_i)$ per index.
- **Layer 2** ($2n = 8$): $\text{won}_i = \text{relu}\Big(\sum_j \big(\text{relu}(d_{ij}) - \text{relu}(d_{ij}-1)\big) - (n-2)\Big)$. This vector **is** the one-hot answer. Plus carried x_i.
- **Layer 3** ($n=4$): $a_i = \text{relu}(x_i + M\text{won}_i - M)$; the value is $\sum_i a_i$.

For the pair $(i,j)=(1,0)$ ($j<i$, strict, sign $+1$) layer 1 contains the rows

$$\begin{pmatrix} 1 & -1 & 0 & 0 \\ 1 & -1 & 0 & 0 \end{pmatrix} x + \begin{pmatrix} 0 \\ -1 \end{pmatrix} = \begin{pmatrix} \text{relu}(x_1 - x_0) \\ \text{relu}(x_1 - x_0 - 1) \end{pmatrix}.$$

For the pair $(1,2)$ ($j>i$, non-strict) the biases are $(+1, 0)$ instead and the columns are $(0, 1, -1, 0)$. For argmin, flip the sign of every weight in the x columns; nothing else changes.

Layer 2's row for $i=1$ adds the three beats-bits: $(+1,-1)$ for each of its three pairs, bias $-(n-2) = -2$.



Cost: n^2

Every pair of elements is compared, so layer 1 has $2n(n-1)$ neurons. That is quadratic:

n	pairs $n(n-1)$	layer-1 neurons	layers	note
4	12	28	3	the example
10	90	190	3	
100	9 900	19 900	3	still shallow, but wide

The depth stays at 3 no matter how big n is. Everything is compared in **parallel**; what you pay is width. (Tournament-style `max` of pairs would need only $n - 1$ comparisons but $\log_2 n$ layers, and it makes the one-hot and the tie-break harder.)

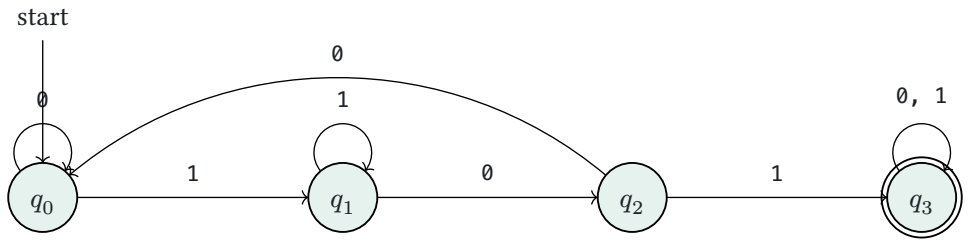
Pitfalls

- **Candidates that must not win.** If some cells are not eligible (e.g. A* nodes not in the open list), do not exclude them with an `if`: add a large constant to their value first (for argmin, add a constant `BIG` where the mask is 0, via a select). Make `BIG` larger than any real value so they can never win, and widen M accordingly.
- **The tie rule is part of the algorithm.** Swap the strictness and you get the *last* maximum instead; both are valid but only one matches the reference program.
- **Integers only.** The `step(d)` and `step(d + 1)` trick for `>` and `>=` is exact only for integers. Use scaled integer costs (e.g. multiples of 10), not floats.

13 State machine

basic_python/03_memory/13_state_machine.py

Problem. Does a bit string contain the substring `101`? Strings have length 0 to 6. A finite automaton (DFA) solves this with four states; here it is as a network.



q_0: nothing useful yet. q_1: just saw `1`. q_2: just saw `10`. q_3: saw `101` (accepting, never leaves).

A · control flow

```
ZERO = {0: 0, 1: 2, 2: 0, 3: 3}
ONE  = {0: 1, 1: 1, 2: 3, 3: 3}

def control_flow(s):
    state = 0
    for ch in s:
        table = ZERO if ch == "0" else ONE
        state = table[state]
    return 1 if state == 3 else 0
```

A loop as long as the string, and two table lookups.

B · branchless arithmetic

```
state = [1, 0, 0, 0]        # one-hot
for t in range(L):          # L = 6, fixed
    is_sym = [equals(codes[t], k)
              for k in range(K)]
    new = [0] * S
    for k in range(K):
        for s_next in range(S):
            moved = T[k][s_next] @ state
            new[s_next] += relu(
                moved + is_sym[k] - 1)
    state = new
```

Step 1: the state becomes a one-hot vector

A state is a number 0..3; we store it as a vector with a single 1: $q_2 = (0, 0, 1, 0)$. "Which state am I in?" is now an address, exactly like in example 10.

Step 2: one constant matrix per symbol

For each symbol k build a constant 4×4 matrix T_k with $T_{k[s',s]} = 1$ when the DFA goes from state s to state s' on symbol k. Then T_k times a one-hot state is the one-hot of the next state, since multiplying a matrix by e_s picks out column s.

The code has three symbols: `0`, `1`, and a padding symbol (code 2) that fills strings shorter than 6 with "do nothing", so every input has the same length. Rows are the **next** state, columns the **current** state.

$$T_0 = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \qquad T_1 = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix} \qquad T_{\text{pad}} = I = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

on `0`: $q_0 \to q_0, q_1 \to q_2, q_2 \to q_0, q_3 \to q_3$ — on `1`: $q_0 \to q_1, q_1 \to q_1, q_2 \to q_3, q_3 \to q_3$ — padding: stay

Read the matrices column by column against the diagram: column 1 of T_0 is $(1, 0, 0, 0)$ (q_0 on a `0` goes to q_0), column 2 is $(0, 0, 1, 0)$ (q_1 on `0` goes to q_2), and so on. Each column contains exactly one 1, because the automaton is deterministic. That property matters below.

Step 3: use the symbol that was actually read

Which matrix applies depends on the runtime symbol, so we cannot just pick one. Instead we apply **all three** and keep the right result using a 0/1 flag $\text{is}_k = [c_t = k]$ (computed with the equality tent from example 10):

$$\text{next} = \sum_k \text{AND}(\text{is}_k, T_k \text{ state}) = \sum_k \text{relu}(T_k \text{ state} + \text{is}_k - 1).$$

Everything here is a 0/1 bit, so the AND is exact. Only the term whose $\text{is}_k = 1$ can be nonzero; for the others T_k state $+ 0 - 1 \le 0$ and the ReLU zeroes them. No big-M is needed because nothing is a large number: both factors are bits.

Trace, string `0101`, padded to `0101`$\cdot\cdot$ (codes $0, 1, 0, 1, 2, 2$):

step t	symbol	state before	the winning term	state after
0	`0`	q_0	$T_0 e_0 = e_0$	q_0
1	`1`	q_0	$T_1 e_0 = e_1$	q_1
2	`0`	q_1	$T_0 e_1 = e_2$	q_2
3	`1`	q_2	$T_1 e_2 = e_3$	q_3
4	pad	q_3	$I e_3 = e_3$	q_3
5	pad	q_3	$I e_3 = e_3$	q_3

At the end, accepting is a dot product: accept $= (0, 0, 0, 1) \cdot$ state, giving 1.

At step 1 in detail (state e_0, symbol `1`, so is $= (0, 1, 0)$): $k = 0$: $\text{relu}(T_0 e_0 + 0 - 1) = \text{relu}(e_0 - 1) = 0$. $k = 1$: $\text{relu}(T_1 e_0 + 1 - 1) = \text{relu}(e_1) = e_1$. $k = 2$: $\text{relu}(I e_0 - 1) = 0$. Sum: e_1.

> **Why determinism matters**
>
> Each column of every T_k has exactly one 1, so T_k state is again a one-hot (entries 0 or 1, never 2). If two states could move to the same target the product would still be a bit, but a *non*-deterministic automaton (one state going to two targets) could create sums > 1 and you would need an OR (`step`) after the AND. The DFA avoids that.

The network: one layer, repeated

Concatenate the state with a "window" holding the not-yet-read flags: $h = (\text{state} \mid \text{is}_{0,0..2} \mid \text{is}_{1,0..2} \mid \ldots)$, 4 + 18 entries. Then **one** step layer, with the **same** weights each time:

$$\text{hidden} = \text{relu}(Ah + b), \quad h' = B \text{ hidden}.$$

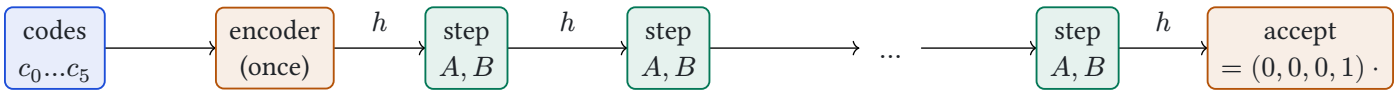
- Rows (s, k) of A: [row s of T_k | pick the first window flag for symbol k] with bias -1. That is exactly T_k state $+ \text{is}_k - 1$ for output s.
- Extra rows of A carry the window forward unchanged (flags are ≥ 0 so ReLU is exact).
- B sums over k to get the new state, and **shifts the window left** by one symbol so that the next character's flags are at the front. After $L = 6$ shifts all flags were used; the zero fill is never read.

A tiny slice: the three hidden rows for next-state q_1 (state columns first, then the first window position's flags for `0`, `1`, pad):

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \text{state} \\ \text{is}_0 \\ \text{is}_1 \\ \text{is}_{\text{pad}} \end{pmatrix} + \begin{pmatrix} -1 \\ -1 \\ -1 \end{pmatrix}$$

(First row: T_0's row 1, the third: I's row 1.) B adds these three outputs together to form the new state's q_1 component.

The **encoder** that creates the flags runs once: for each position t and symbol k, four ReLUs $\text{relu}(c_t - k), \text{relu}(c_t - k - 1), \text{relu}(k - c_t), \text{relu}(k - c_t - 1)$ and then $\text{is} = 1 - r_0 + r_1 - r_2 + r_3$ (72 neurons).



same weights, 6 times

Cost and trade-offs

Each step layer has $SK + LK = 12 + 18 = 30$ neurons; the whole network is the encoder (72) plus six identical steps. A longer string needs more repetitions, so the maximum length is fixed in advance, as in chapter 2's bounded loops. A bigger alphabet or more states makes the T_k bigger, but the structure stays the same. Padding with an identity symbol is how a fixed-depth network handles shorter inputs.

Try it

1. **(10)** What does the read network return for $i = 7$ (outside $0..3$)? Why is that dangerous?
2. **(11)** Write `xs[i] += 1` as a network. Do you need the big-M select?
3. **(12)** Change the tie-break so that the **last** maximal index wins.
4. **(13)** Design a DFA for "contains `11`" and give `DELTA` and the accepting set.

Answers

1. **(10)** Every tent $[i = j]$ is 0 for $i = 7$, so all four a_j are 0 and $y = 0$. No error is raised; an invalid address silently reads as 0. If 0 is also a legitimate stored value you cannot tell the difference, so make sure addresses stay in range, or reserve a value for "invalid".

2. **(11)** No select is needed: $\text{new}_j = \text{xs}_j + [i = j]$. The added amount is a constant (1) multiplied by a bit, which is just the bit. In the network, layer 1 computes the tents as before, and the output layer is affine: $\text{new}_j = \text{relu}(\text{xs}_j) + (1 - r_0 + r_1 - r_2 + r_3)$. Two layers of weights and no M. (Selects are only needed when the written value is itself a runtime quantity.)

3. **(12)** Swap the strictness: strict ($\text{step}(d)$) against **later** indices ($j > i$), non-strict ($\text{step}(d + 1)$) against **earlier** ones ($j < i$). Then on a tie the later index wins every contest. In the code: `step(d) if j > i else step(d + 1)`; in the weights, swap which pairs get bias $c = 0$ and which get $c = 1$.

4. **(13)** Three states: q_0 nothing, q_1 just saw `1`, q_2 saw `11` (accepting, stays). `DELTA = {0: [0, 0, 2], 1: [1, 2, 2], 2: [0, 1, 2]}` (rows: symbol `0`, symbol `1`, pad; entries: next state for q_0, q_1, q_2). Accepting vector $(0, 0, 1)$. Set `S = 3`; T_k are rebuilt from `DELTA` automatically. Nothing else in the network changes: the step layer is regenerated with $3K + LK$ rows.